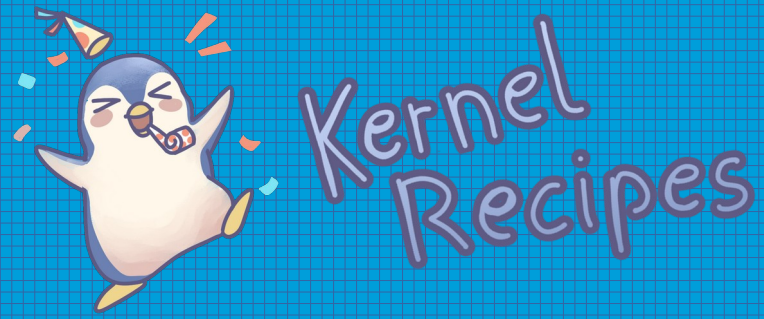
Improving Linux for (KVM) Real Time Workloads

Leonardo Brás Soares Passos
Kernel Recipes 2025



whoami

- Leonardo Brás Soares Passos
- Work @ Arm UK (started this month)
  - Kernel team
    - arm64
    - KVM
- Previously: Virt-RT team @ Red Hat
- Find me: leobras @ {IRC, GitLab, GitHub, Mastodon}



Disclaimer

This presentation is based in the experience acquired during my employment at Red Hat, as a member of the Virt-RT team.

This work was not done in any capacity of my Arm employment.

Part 1:
The basic stuff
(for Real-Time)

What is Real Time ?

What is Real Time ?

I need you to deal with my request in T time, or else…

What is Real Time ?

I need you to deal with my request in T time, or else...

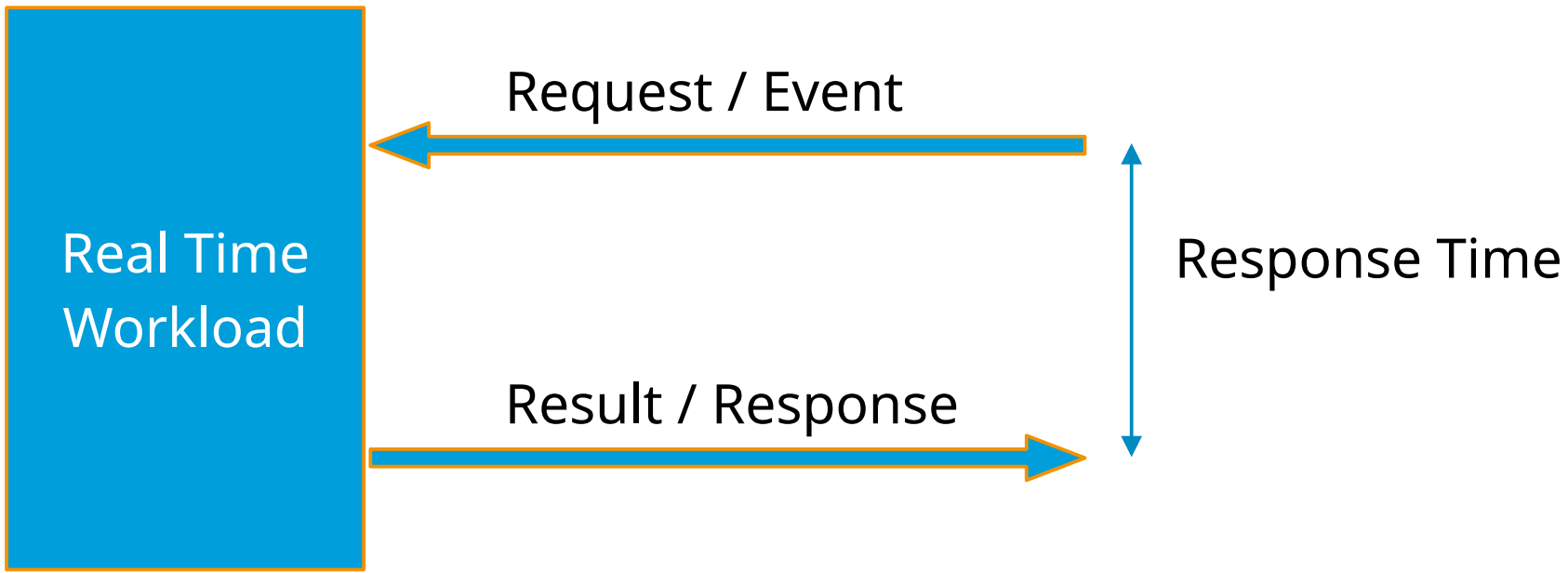


What is Real Time ?

- Real-time (computing) is the computer science term for hardware and software systems subject to a "real-time constraint", for example from event to system response.[1]

- Real-time programs must guarantee response within specified time constraints, often referred to as "deadlines". [2]

What is a deadline?



Deadline = Maximum “Response time” acceptable by given workload

Why does Real Time Matter?

- Missing a deadline can have bad consequences [3]:
  - Hard – missing a deadline is a total system failure.
  - Firm – infrequent deadline misses are tolerable, but may degrade the system's quality of service. The usefulness of a result is zero after its deadline.
  - Soft – the usefulness of a result degrades after its deadline, thereby degrading the system's quality of service.

Real Time workload examples

- Hard Real Time:
  - Medical Systems (pacemakers, robot surgery)
  - Industrial Process Controllers (assembly line, often Firm RT)

- Soft Real Time:
  - Live audio / video systems (such as online meeting apps)
  - Video games (missing deadline degrades experience)

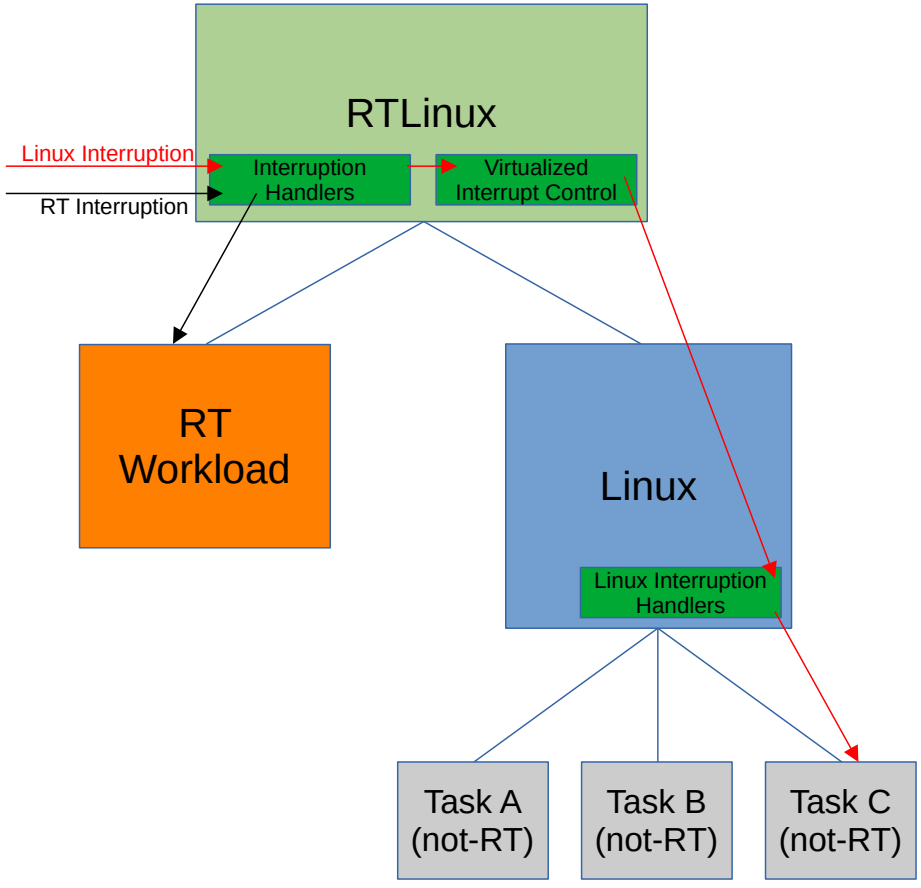
Real Time Operating Systems

- Historically Real Time applications ran on bare metal
  - No Operating Systems bellow
  - To avoid the latency overhead of an operative system
- Programming an application on top of an Operating System is much more comfortable & fast.
  - Abstractions, drivers, libraries
- What about an OS focused on Real Time workloads?

Real Time Operating Systems

- In order to have a RTOS, a different strategy is needed:
  - Zephyr: Simpler kernel, with RT-oriented mechanisms. Many Schedulers available for different RT requirements . [4]
  - FreeRTOS: Whole OS is deterministic, has many memory allocation options (for different RT requirements), don’t share stack between processes [5]
  - RTLinux: Linux is ran for non-RT tasks, relying on a virtualized interrupt control. RT tasks run on a POSIX thread on RTLinux infrastructure (not using Linux infrastructure).
    - RT threads have higher priority compared to Linux threads [6]

RTLinux


RTLinux
Linux Interruption
RT Interruption
Interruption Handlers
Virtualized Interrupt Control
RT Workload
Linux
Linux Interruption Handlers
Task A (not-RT)
Task B (not-RT)
Task C (not-RT)

What about Linux ?

- Linux is the major free OS in the market
  - But was not planned as a Real Time operating system.
- Uses performance-oriented schedulers
  - Focus on throughput instead of latency
  - Makes sense on server workloads
- No deterministic behavior
  - Workloads can be interrupted by scheduler or kernel
    - possibly causing missed deadlines.
- What if we turn Linux on a Real Time OS?

Turn Linux on a Real Time OS?

- As one of the major OS
  - Many programming languages, libraries & tools are available
- An extensive hardware support library (drivers)
  - More options during hardware design
- Source available
  - Code can be modified to add new feature or device support

Part 2:
Improving Real-Time behavior on Linux Kernel

Real-Time scheduling policies

Real-Time scheduling policies

Run my task first!

Real-Time scheduling policies

Run my task first!



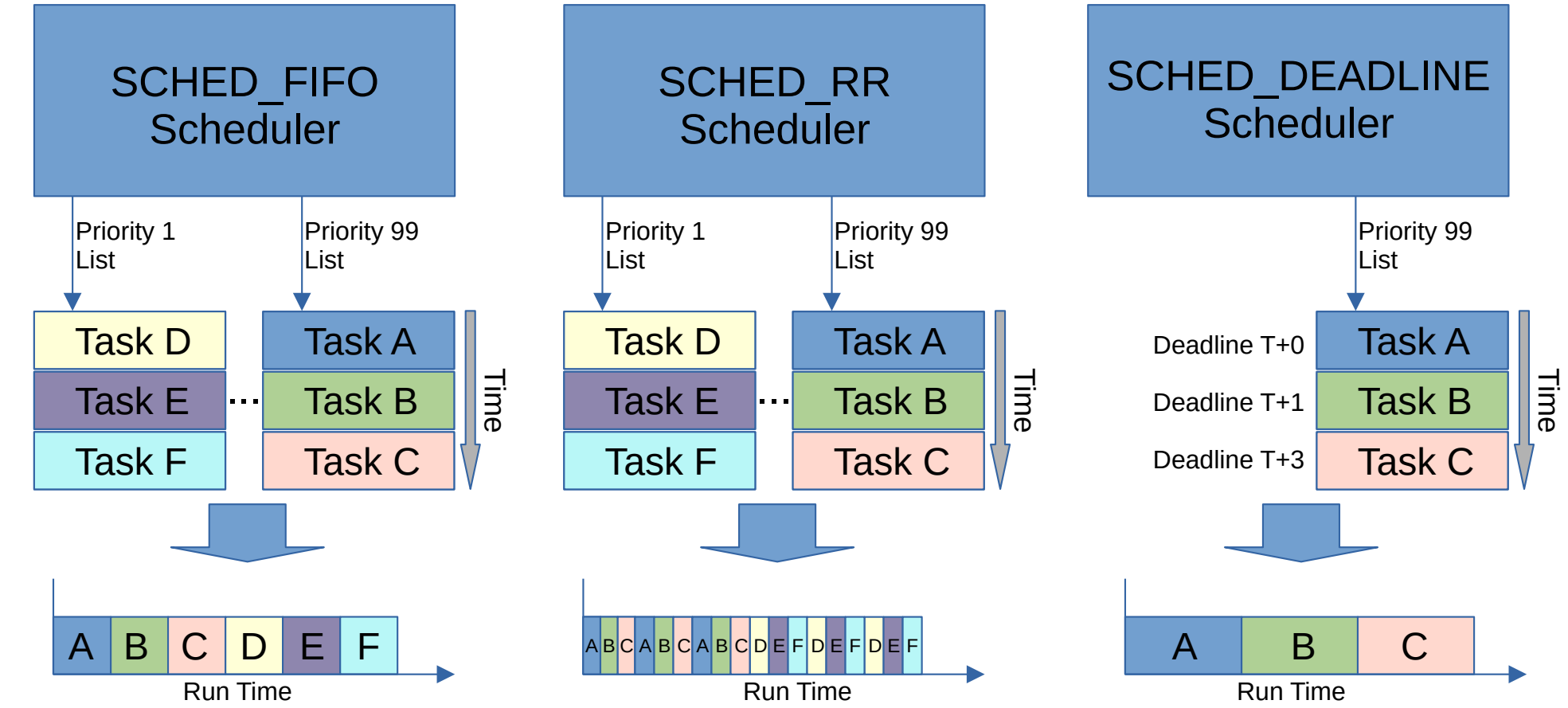
Real-Time scheduling policies

- The scheduler is the kernel component that decides which runnable thread will be executed by the CPU.
- Some of those are normal (performance) scheduling policies (SCHED_OTHER, SCHED_BATCH, SCHED_IDLE)
- Other are known as Real-Time scheduling policies
  - They make sure highest priority workloads run first

Real-Time scheduling policies

- SCHED_FIFO
  - Highest priority task will always run first, until finished
  - Scheduler picks task which was scheduled first for running next.
    - Given same priority
- SCHED_RR (Round-Robin)
  - Scheduler run a highest priority task for a time period, and switches to the other same-priority task after that.
- SCHED_DEADLINE
  - Runs the task with the earliest deadline first
  - Need to specify Runtime, Deadline & Period

Real-Time scheduling policies



PREEMPT_RT

PREEMPT_RT

Stop the kernel, run my task!

PREEMPT_RT

Stop the kernel,
run my task!



PREEMPT_RT

What is it?

- A kernel option that enables the preemption model:
  - "Fully Preemptible Kernel (Real-Time)"
    - “This option turns the kernel into a real-time kernel”
  - What is Preemption?
    - Interrupting a task in order to execute a higher priority task
  - tl;dr: Allow interruption of Kernel code by userspace code

PREEMPT_RT

What does it change?

- Replace locking primitives (spinlocks, rwlocks, etc.)
  - Locks turned on preemptable, priority-inheritance aware variants
  - Serve as mechanisms to break long non-preemptable sections
- Allow the kernel to be (mostly) preemptable

- Some code paths such as ‘entry code’, scheduler, and low level interrupt handling are still not-preemptable.

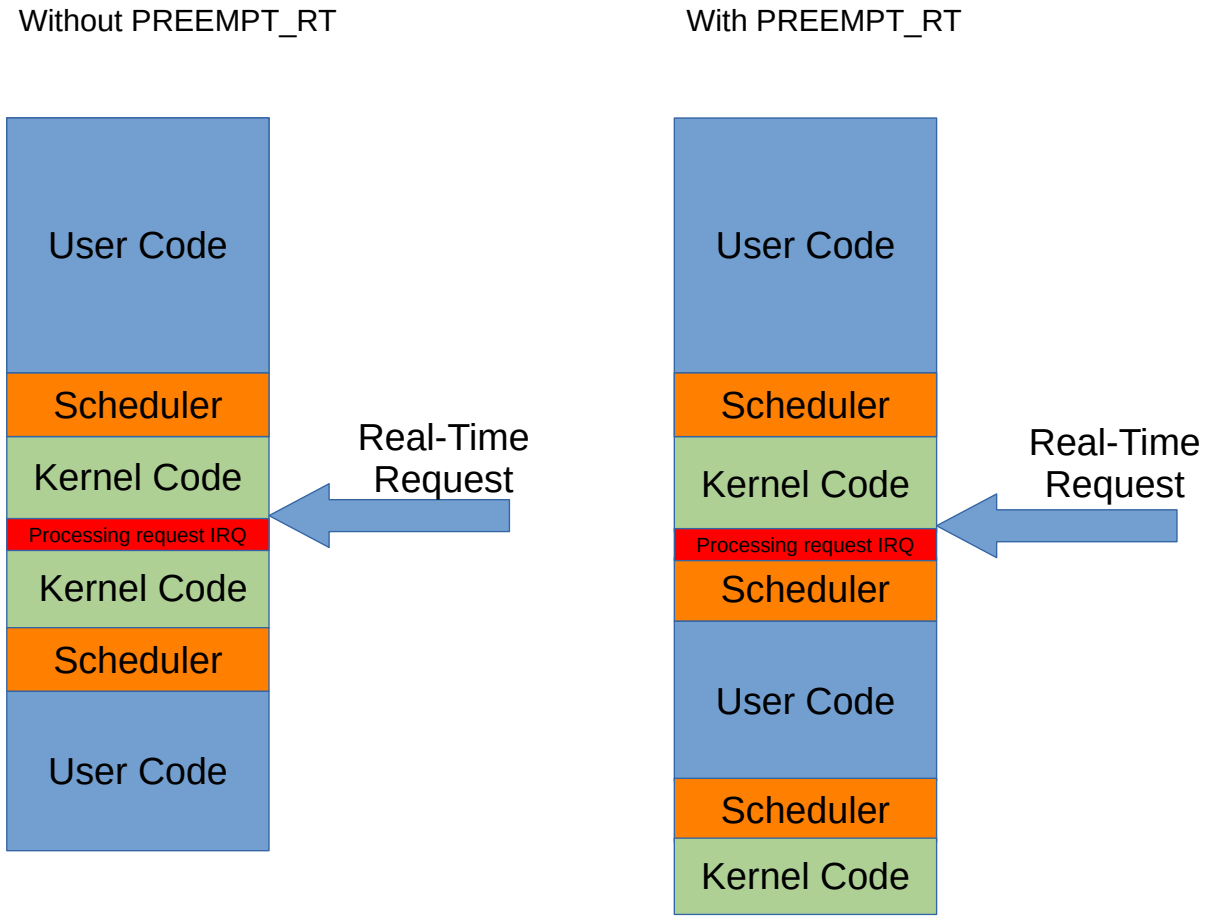
PREEMPT_RT

How does it improve RT behavior?

- Preemptable kernel → kernel can be “interrupted”

- RT workload gets higher priority → Gets CPU time earlier
  - RT workload has reduced latency
  - Also has reduced chance of missing a deadline

PREEMPT_RT


Without PREEMPT_RT
User Code
Scheduler
Kernel Code
Processing request IRQ
Kernel Code
Scheduler
User Code
Real-Time Request
With PREEMPT_RT
User Code
Scheduler
Kernel Code
Processing request IRQ
Scheduler
User Code
Scheduler
Kernel Code
Real-Time Request

CPU Isolation

CPU Isolation

Don’t run stuff at these CPUs!

CPU Isolation

Don’t run stuff at these CPUs!



CPU Isolation

What is it?

- Linux boot parameter
  - Remove a CPU list from the SMP load balancer and scheduler.
  - Tell other Linux code to avoid scheduling work on it.

CPU Isolation

What does it change?

- Scheduler don’t schedule work on Isolated CPUs by default
  - User need to manually assign (pin) a process to given CPU
    - User has full control of what runs there.

- Also avoid some ‘housekeeping’ kernel work there
  - Less workload interruption

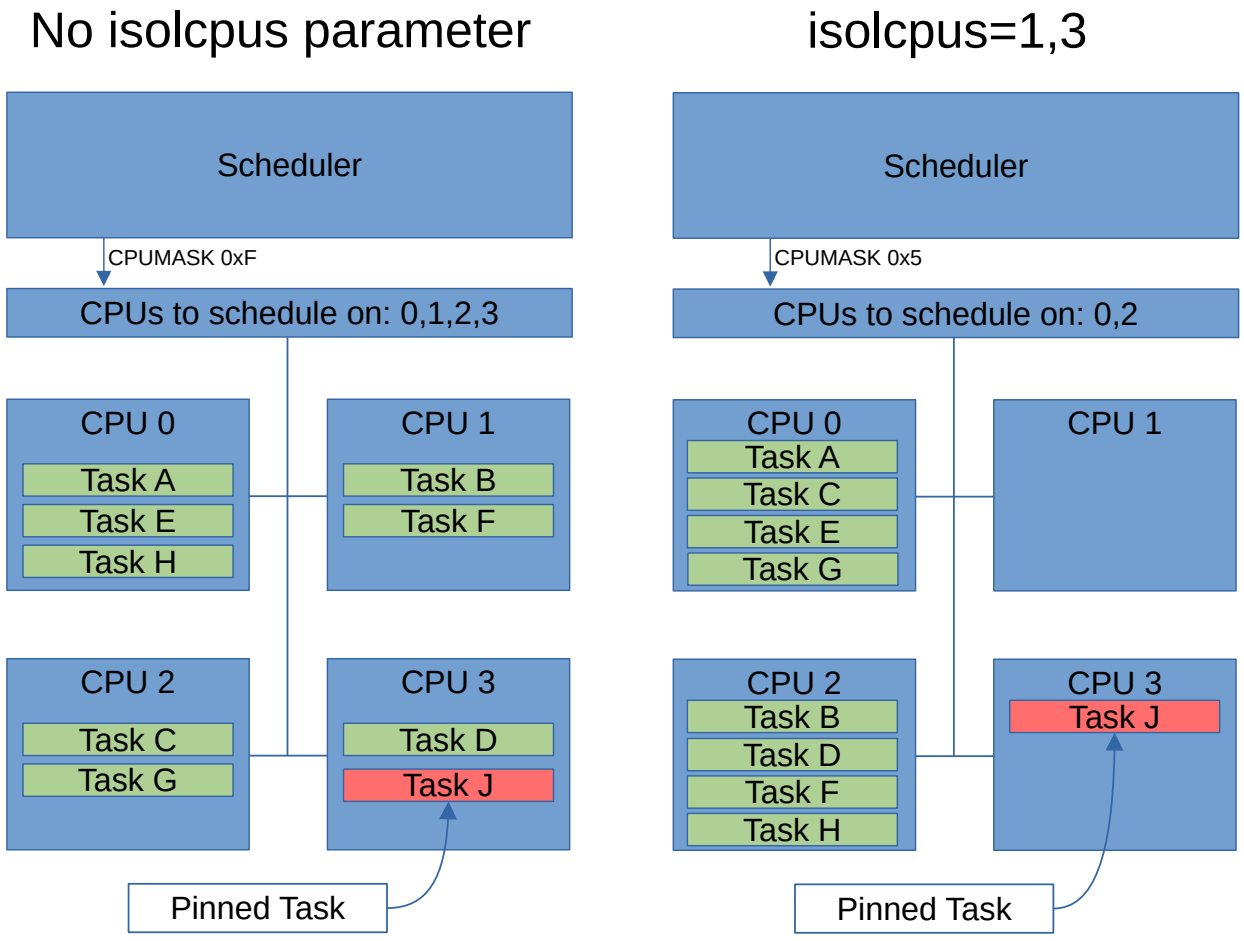
CPU Isolation

How does it improve RT behavior?

- RT workloads can be ‘pinned’ to Isolated CPUs

- Single workload per CPU → Not competing for CPU time
  - Multiple CPUs can handle multiple different RT-workloads
  - Non-RT stuff can run on other CPUs

CPU Isolation



NOHZ_FULL

NOHZ_FULL

Is my task alone?

Don’t run the scheduler!

NOHZ_FULL

Is my task alone?

Don’t run the scheduler!



NOHZ_FULL

What is it?

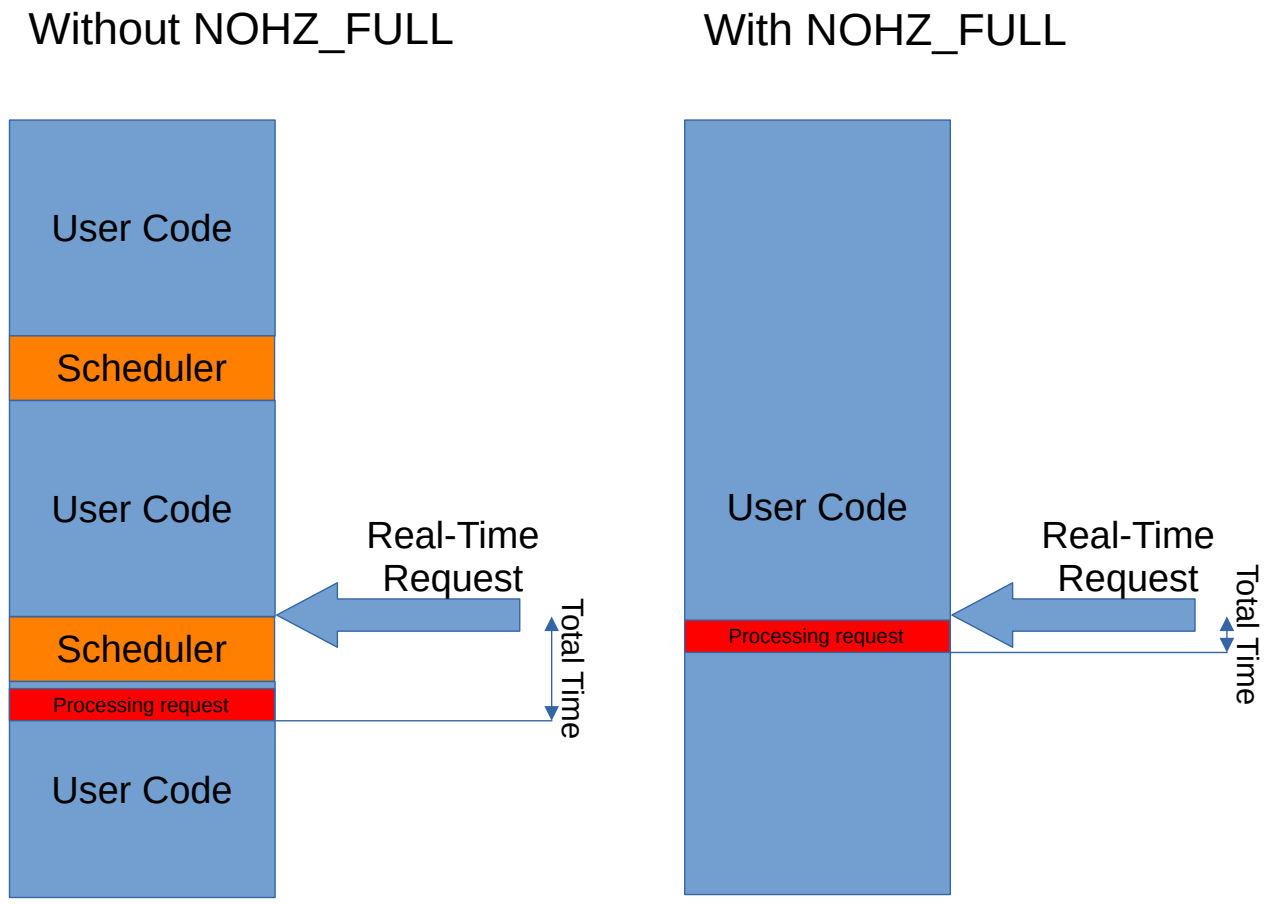
- Linux boot parameter
  - Takes a cpu-list
  - Turns-off the tick for a CPU (when there is a single task).
  - Also offloads RCU callbacks
    - to CPUs that are not in the NOHZ_FULL list
- Requires CONFIG_NO_HZ_FULL=y

NOHZ_FULL

What does it change?

- Less interruption
  - No scheduler interrupting CPU to check for other tasks
  - RCU callbacks will not be ran in that CPU
    - (Offloaded to other CPUs)
- How does it improve RT behavior?
  - Less interruption → RT workload gets more CPU time

NOHZ_FULL


Without NOHZ_FULL
With NOHZ_FULL
User Code
Scheduler
User Code
Scheduler
Processing request
User Code
Real-Time Request
Total Time
User Code
Processing request
Real-Time Request
Total Time

Part 3: Numbers



Tests

- Cyclictest
  - “Sleeps” for given time, checks maximum time spent after return.
  - Good for checking predictability
- Oslat
  - Reads time in a loop, checks largest value between
  - Measures the maximum time userspace get interrupted
- Stress-ng
  - Creates cpu/mem workload to try and mess latency

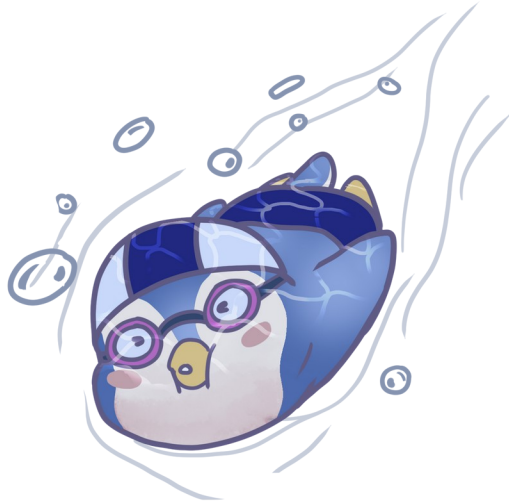
Numbers

- Cyclictest: 12h, 20CPUs
  - No other workload
    - Average: 4.57us, Higher: 8us
  - Workload in other CPU
    - Average: 4.7us, Higher: 8us
  - Workload in same CPU
    - Average: 31us, Higher: 34us

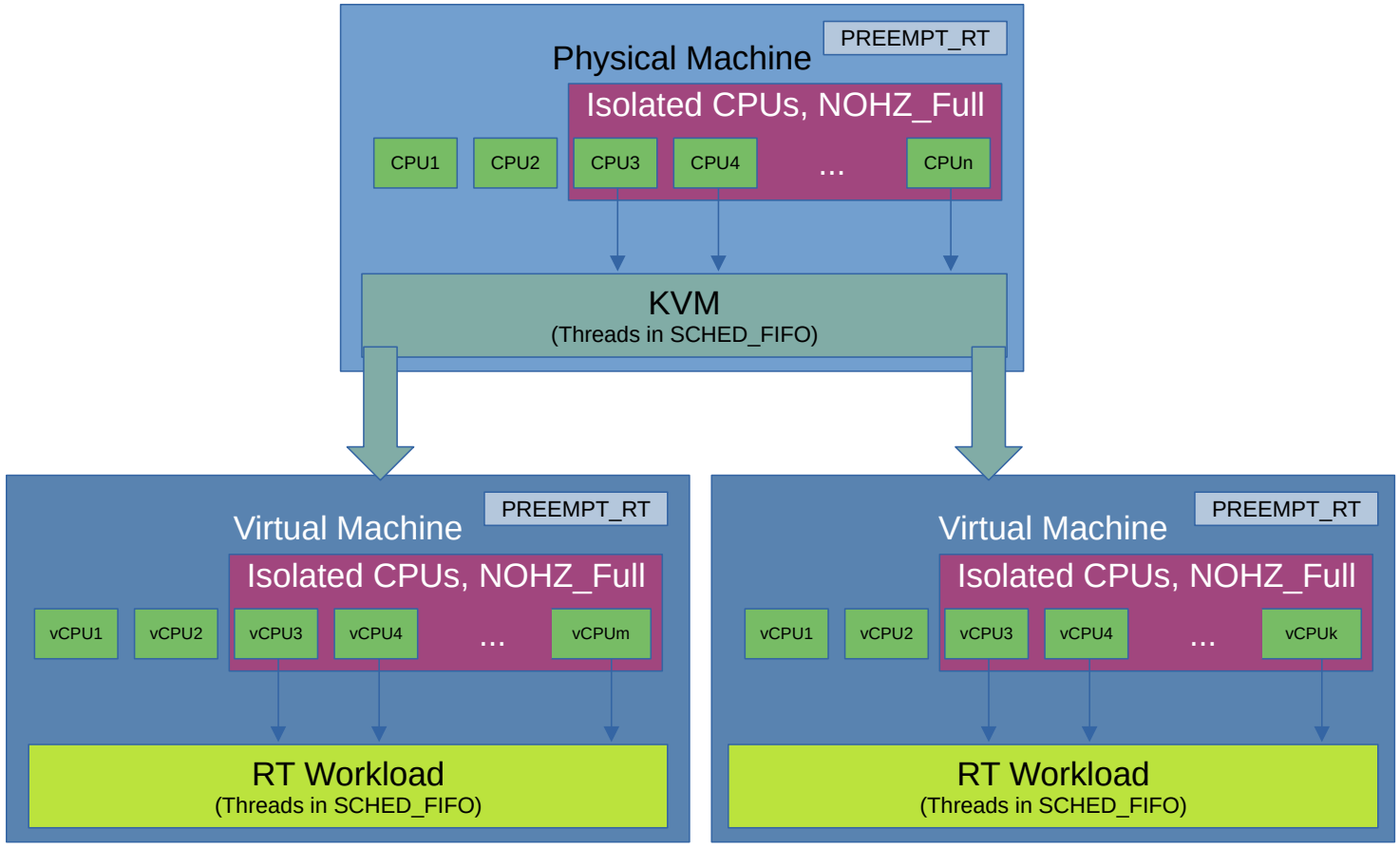
Numbers

- Oslat: 12h, 20CPUs
  - No other workload
    - Average: 2us, Higher: 2us
  - Workload in other CPU
    - Average: 2.1us, Higher: 3us
  - Workload in same CPU
    - N/A
    - Oslat does not share cpus well

Part 4:
RT in Virtual Machines



Real Time Virtual Machines



Motivation

Same as regular VM motivations.

- Can use fewer, bigger & more efficient machines
  - While meeting the latency needs
  - Centralizing workloads
- Easier to move workload when needed (Live Migration)
  - More modular approach
- Cloud providers could rent Real-time Virtual machines
  - Easier to implement RT workloads
  - No need to worry on hardware specs

Drawbacks

- Virtualization adds overhead, and thus adds latency
  - guest_exit and guest_entry take time
  - Preemption may need to happen both on guest and host
  - Housekeeping tasks both in guest and host
- Deadline need to cover the network latency
  - In case the processing is far from the user

Numbers

- Oslat: 12h, 16CPUs
  - Stress on non-isolated cores at host & guest
  - Average: 14us, Higher: 23us
- Cyclictest: 24h, 8CPUs
  - Stress on non-isolated cores at host & guest
  - Average: 15.6us, Higher: 18us

Numbers

- Oslat: 12h, 16CPUs
  - Stress on non-isolated cores at host & guest
  - Average: 14us, Higher: 23us
- Cyclictest: 24h, 8CPUs
  - Stress on non-isolated cores at host & guest
  - Average: 15.6us, Higher: 18us

Virt-RT team works on further reducing those numbers

Some of my work on RT

- Add tracepoints on remotelly scheduled functions
- Avoid isolated cpus for queue_delayed_work()
- Note RCU quiescent state in guest_exit
  - Avoid rcu_core() running short after guest_exit
  - Convinced paulmck to add new “patience” rcu option
- Change local_lock strategy in RT to avoid IPIs
  - Ongoing, should remove a lot of IPIs for Isolated CPUs



Thanks!

Questions?

References:

[1] "FreeRTOS - Open Source RTOS Kernel for small embedded systems - What is FreeRTOS FAQ?". FreeRTOS. Retrieved 2021-03-08.

[2] Ben-Ari, Mordechai; "Principles of Concurrent and Distributed Programming", ch. 16, Prentice Hall, 1990, ISBN 0-13-711821-X, page 164

[3] Kopetz, Hermann; Real-Time Systems: Design Principles for Distributed Embedded Applications, Kluwer Academic Publishers, 1997

[4] “Zephyr Documentation : Schedulers”. Retrieved 2023-08-23 https://docs.zephyrproject.org/latest/kernel/services/scheduling/index.html

[5] “FreeRTOS Documentation”. Retrieved 2023-08-23. https://www.freertos.org/features.html

[6] “RTLinux page”. Retrieved 2023-08-23. https://en.wikipedia.org/wiki/RTLinux

[7] “Linux Kernel Documentation”. Retrieved 2023-08-24. https://www.kernel.org/doc/html/latest